

AI Intern — Technical Assessment

Agentic CRM Intelligence Platform & Real-Time Email Operations System

Duration	2–3 days (take-home)
Dataset	email-data-advanced.json (60 emails, 30 threads)
Stack	Your choice — justify your decisions in the README
Deliverable	GitHub repo + architecture diagram + screen recording
Bonus Tracks	RAG pipeline · Autonomous Agent · Live Web Intelligence · Sentiment Trend Dashboard

Important: This test is intentionally open-ended. There is no single correct architecture. We are evaluating your reasoning, trade-off analysis, and ability to build production-quality AI systems — not your ability to follow a recipe. Please complete as much as possible within the time frame — if you complete everything that is excellent, but full completion is not a necessary selection criteria. We are more interested in the quality of what you build than the quantity.

Core Philosophy

You are designing a production system for a real AI solutions company. Every architectural decision you make — from database schema to LLM prompt design — should be defensible. Document your trade-offs. Handle failure modes. Think about what breaks at scale.

1. Project Overview [CORE]

Build a production-grade, AI-powered Customer Relationship Management (CRM) system capable of autonomously monitoring a high-volume inbox, triaging emails with multi-dimensional intelligence, executing agentic workflows, and surfacing real-time business insights. The system must handle the full email lifecycle from ingestion to resolution — including edge cases, conflicting signals, and ambiguous intent.

What makes this 'advanced':

- The AI must act as an **autonomous agent** — not just classify, but *decide and act* across multi-step workflows.
- A **RAG pipeline** grounds the agent in your internal knowledge base (pricing docs, SLA policies, product FAQs).
- A **live web intelligence** module scrapes public signals (review sites, competitor pricing, social mentions) and surfaces them in context.
- Emails form **conversation threads** — the agent must read the full thread history before acting.
- The system must handle **conflicting signals**, **escalation chains**, and **GDPR/compliance** edge cases gracefully.

2. Dataset: email-data-advanced.json [DATA]

The provided JSON file contains 60 realistic emails across 30+ named conversation threads. Key scenarios that your system must handle correctly:

- `thread_alice_pricing` — Multi-turn pricing negotiation → upgrade → billing question (5 emails across 13 days)
- `thread_bob_outage` — P0 incident → SLA breach → RCA → legal escalation (4 emails, high-stakes)
- `thread_karen_refund` — Complaint with 0 replies → escalating anger → churn threat → public review threat
- `thread_eleanor_compliance` — Enterprise HIPAA compliance deal — 200 seats, decision-deadline pressure
- `thread_bigcorp_rfp` — RFP + compliance audit + new user registration — all linked to one \$2.4M opportunity
- `thread_security_*` — Two security threats: suspicious login + ransomware — require immediate escalation
- `thread_gdpr_001` — Formal GDPR Article 20 data portability request — legal obligation, time-boxed
- `thread_spam_*` — Multiple spam categories: SEO pitch, Nigerian prince, cold outreach — must not auto-reply
- `thread_nadia_bug` — Silent data corruption bug — success message shown but data missing, mission-critical
- `thread_chatbot_misinformation` — Your own chatbot gave wrong refund info — trust and liability issue

Threading requirement: Emails are linked by `thread_id` and sender. Your system must group by thread and provide the full conversation context to the AI before any classification or response decision is made.

3. Component 1 — Email Ingestion & Streaming Pipeline [REQUIRED]

Objective:

Simulate a realistic real-time email stream with deduplication, schema validation, and graceful handling of malformed payloads.

Required:

- **POST /api/ingest** — Accepts JSON email payload; validates schema; rejects malformed input with descriptive errors
- **Streaming simulation** — Replay email-data-advanced.json at configurable speed (e.g., 1 email/sec in dev, 10/sec in load test)
- **Deduplication** — Idempotent ingestion: re-sending the same message_id must not create duplicates
- **Thread linking** — On ingest, automatically link to existing thread or create a new one
- **Priority queue** — Assign initial priority score on ingest based on keyword heuristics before AI processing

Edge cases you must handle:

- Empty body or subject
- Body with only whitespace or HTML entities
- Duplicate message_id (re-delivery simulation)
- Timestamps out of order within a thread
- Extremely long body (>10,000 characters) — truncate or chunk for LLM processing

4. Component 2 — Multi-Layer Intelligence Engine [REQUIRED + BONUS]

Layer 1 — Heuristic Pre-filter (Required)

Run synchronously on ingest for immediate triage. Must be fast (sub-10ms).

- Spam detection: keyword blocklist + sender domain reputation
- Urgency detection: words like 'URGENT', 'PO', 'legal', 'cease and desist', 'ransomware'
- Security flag: detect login alerts, data breach threats, and route immediately to security queue
- Internal email filter: route @internal.com and @mycompany.com to separate inbox

Layer 2 — LLM Classification Engine (Required)

Use an LLM API to classify each non-spam, non-internal email. The prompt must include: (a) the full thread history, (b) relevant RAG context (see Component 4), and (c) structured output instructions.

Required output schema (JSON):

```
{ "category": "Complaint|Inquiry|Bug Report|Feature  
Request|Compliance|Legal|Billing|Spam|Internal|Other", "sentiment":  
"Positive|Neutral|Negative|Mixed", "sentiment_score": -1.0, // float: -1.0 (very negative)  
to +1.0 (very positive) "urgency": "Critical|High|Medium|Low", "requires_human": true, //
```

```
boolean "escalation_reason": "...", // string if requires_human=true, else null
"suggested_reply": "...", // string if requires_human=false, else null "confidence": 0.91,
// float: 0.0 to 1.0 "detected_entities": { // named entity extraction "order_ids": [],
"ticket_ids": [], "monetary_amounts": [], "deadlines": [], "products_mentioned": [] } }
```

Challenge — Conflicting Signal Handling:

Some emails contain conflicting signals (e.g., 'I love the product but hate the price and want a refund'). Your system must resolve conflicts and document the resolution strategy in your README. A confidence score below 0.70 should automatically flag the email for human review.

Layer 3 — Sentiment Trend Tracking (Required)

- Track sentiment_score over time per sender (moving average)
- Detect sentiment deterioration: 3+ consecutive negative emails from one sender → escalation alert
- Expose trend data via GET /analytics/sentiment-trend?sender=X&days;=30

5. Component 3 — RAG Knowledge Pipeline [ADVANCED]

The intelligence engine must be grounded in internal knowledge, not just LLM parametric memory. Build a Retrieval-Augmented Generation pipeline that fetches relevant policy/product context before the LLM drafts any response.

Knowledge Base (you must create these documents):

Document	Contains
pricing_policy.md	Pricing tiers, non-profit discounts (30% off Standard), pro-rata billing rules, enterprise custom pricing
sla_policy.md	Uptime SLA (99.9%), incident response times, credit calculation formula, RCA delivery SLA (24h for P0)
refund_policy.md	No refunds after 14 days, exception process, credits vs refunds, churn retention playbook
api_docs.md	Rate limits by tier, v1 deprecation timeline, v2 breaking changes, header requirements
compliance_faq.md	HIPAA BAA availability, GDPR DPA process, SOC 2 Type II status, data residency options
escalation_matrix.md	Who handles: legal threats, security incidents, PR crises, VIP churns, GDPR requests

RAG Implementation Requirements:

- Chunk documents into 300–500 token segments with overlap
- Embed chunks using any embedding model (OpenAI, Cohere, sentence-transformers, etc.)
- Store vectors in a vector database of your choice (Pinecone, Chroma, Weaviate, pgvector, FAISS)
- On each email classification, retrieve top-3 relevant chunks and inject into LLM context
- The LLM prompt must cite which policy document informed the suggested reply
- Expose GET /rag/search?q=... for debugging — show retrieved chunks and similarity scores

Evaluation scenario:

When karen.w@retail-co.com sends 'I want a refund' — the RAG pipeline must retrieve the refund policy, the retention playbook, AND the escalation matrix (because she's threatened public reviews). The LLM must synthesize all three and produce an appropriate escalation recommendation, not just a generic 'we'll look into it' reply.

6. Component 4 — Autonomous Triage Agent [ADVANCED]

Build a multi-step autonomous agent that can take actions — not just classify. The agent must reason across tools and the thread history to decide the best course of action.

Agent Tools (implement at least 4):

- `search_knowledge_base(query)` — RAG search across internal docs
- `get_thread_history(sender_email)` — Retrieve all emails from this sender, ordered by time
- `get_contact_profile(email)` — Fetch CRM profile: VIP status, account value, open tickets, churn risk score
- `check_account_status(email)` — Billing status, subscription tier, overdue invoices
- `draft_reply(context, tone, policy_refs)` — Generate a contextual reply citing specific policies
- `escalate_to_human(email_id, reason, priority)` — Route to human with a pre-filled brief
- `create_internal_ticket(title, body, assignee)` — Create a support/engineering ticket
- `scrape_public_sentiment(company_name)` — Web intelligence: check G2/Trustpilot score (see Component 5)
- `flag_for_legal(email_id, issue_type)` — Route legal threats to legal team with context summary
- `send_auto_reply(email_id, draft_id)` — Approve and send an auto-reply

Agent Loop Requirements:

- The agent must reason step-by-step before acting (Chain-of-Thought or ReAct pattern)
- Each agent run must produce a structured reasoning log (stored in DB) showing: Thought → Action → Observation → Next
- Maximum 6 tool calls per email — if unresolved, escalate to human with reasoning summary
- The agent must not auto-reply to emails marked Critical urgency — always escalate
- Implement a 'dry run' mode where the agent shows its plan but does not execute

Mandatory test case — bob.iones@enterprise.net escalation chain:

The agent receives msg_060 ('Escalation: SLA Breach + Legal Review'). It must: (1) retrieve the full thread history (4 prior emails), (2) search the SLA policy for credit obligations, (3) check Bob's account status (Enterprise tier, renewal on hold), (4) recognize the legal threat, (5) `flag_for_legal()`, (6) draft an empathetic holding reply citing the SLA credit policy, (7) `escalate_to_human()` with a pre-filled brief. Show the full reasoning trace in your demo.

7. Component 5 — Live Web Intelligence Module

[ADVANCED]

When the agent handles reputation-sensitive emails (complaints, churn threats, press inquiries), it must enrich its context with live public intelligence scraped from the web.

Scraping Targets (implement at least 2):

- **G2 / Capterra / Trustpilot:** Current star rating, recent review count, common complaint themes (NLP summary of recent reviews)
- **Competitor Monitoring:** Scrape competitor pricing pages on-demand — surfaces when handling pricing objections
- **Social Listening (Optional):** Reddit mentions, Hacker News posts, or Twitter/X threads mentioning your product
- **Company News (Optional):** Recent press releases or news articles about senders from large companies (e.g. BigCorp)

Technical Requirements:

- Async scraping — must not block the main processing pipeline
- Caching layer — cache scrape results for 6 hours to avoid rate limiting
- Scrape results injected into agent context as a 'Market Intelligence' block
- Expose GET /intelligence/reputation returning current public sentiment summary
- Handle scraping failures gracefully — agent should proceed without web data if scrape fails
- Include robots.txt compliance check before scraping any domain

Trigger conditions:

- Email contains 'review', 'Trustpilot', 'G2', 'Twitter', 'post publicly'
- Sentiment score drops below -0.6
- Category = Complaint AND urgency = High or Critical
- Press or investor inquiry detected

8. Component 6 — Database Design [REQUIRED]

Design a normalized relational schema that supports all system features efficiently. Include an ER diagram in your deliverables.

Required Tables:

contacts

id, email, name, company, status (VIP|Blocked|Active|Churned), account_value, churn_risk_score, created_at, last_contact_at

threads

id, thread_id (from JSON), subject, sender_email, first_seen_at, last_updated_at, status (Open|Resolved|Escalated|Ignored), assigned_to

emails

id, thread_id (FK), message_id, sender, subject, body, timestamp, sentiment_score, category, urgency, requires_human, confidence, raw_entities (JSON), status (Received|Processing|Replied|Escalated|Ignored)

actions

id, email_id (FK), agent_reasoning_log (JSON/TEXT), action_type (Auto-Reply|Escalate|Legal-Flag|Ticket-Created|Ignored), proposed_content, is_approved, approved_by, executed_at

knowledge_chunks

id, source_doc, chunk_text, embedding (vector), created_at

web_intelligence_cache

id, source_url, target_entity, scraped_data (JSON), scraped_at, expires_at

audit_log

id, entity_type, entity_id, action, performed_by (agent|user_id), timestamp, diff (JSON)

Performance requirements:

- GET /threads/contact_email must return full thread with all emails in <100ms for threads up to 50 emails
- Sentiment trend query over 30 days must use appropriate indexing — benchmark and include query plan
- Vector similarity search must return top-3 chunks in <200ms

9. Component 7 — Backend API [REQUIRED]

Method	Endpoint	Description
POST	/api/ingest	Ingest a new email; returns processing job ID
GET	/api/status/{job_id}	Check processing status of an ingested email
GET	/dashboard/stats	Counts: Pending, Replied, Escalated, Critical, Spam filtered
GET	/threads/{contact_email}	Full conversation thread with all emails, actions, and agent logs
POST	/respond/{email_id}	Send a reply; updates status; appends to thread
PATCH	/drafts/{id}	Edit a proposed auto-reply before sending
POST	/drafts/{id}/approve	Approve and send; triggers audit log entry
GET	/analytics/sentiment-trend	Time-series sentiment data per sender or global
GET	/analytics/category-breakdown	Category distribution over configurable date range
GET	/rag/search	Debug endpoint: query KB and return chunks + scores
GET	/intelligence/reputation	Latest scraped public sentiment for company
POST	/agent/dry-run/{email_id}	Run agent in planning mode; return reasoning trace without executing
GET	/audit/{entity_type}/{entity_id}	Full audit history for any entity
GET	/contacts/{email}	Contact profile with churn risk, account value, open threads
PATCH	/contacts/{email}/status	Update contact status (VIP, Blocked, etc.)

Required: All endpoints must return consistent error envelopes with `error_code`, `message`, and `details`. Include OpenAPI/Swagger spec in your repo.

10. Component 8 — Frontend Dashboard [REQUIRED]

View 1: Mission Control Inbox

- Filterable/sortable email list with visual badges: Sentiment (color-coded), Category, Urgency
- Tab system: All | Needs Human | Auto-Replied | Escalated | Spam
- Real-time updates via WebSocket or polling (configurable interval)
- Thread grouping: collapse multiple emails from same sender into a single row showing last activity
- Bulk actions: Mark as Spam, Assign to team member, Archive
- Search: full-text search across subject and body

View 2: Thread Workspace (Detail)

- Left pane: Email content with entity highlights (e.g. monetary amounts, ticket IDs underlined)

- Center pane: Chronological thread timeline with sentiment indicator per message
- Right pane: Contact profile card (VIP status, account value, churn risk score)
- Agent Reasoning Panel: Collapsible view of the agent's Thought → Action → Observation trace
- RAG Context Panel: Show which knowledge chunks were retrieved and their similarity scores
- Action area: 'Approve & Send' / 'Edit Draft' / 'Escalate' / 'Mark Spam' buttons
- If web intelligence was fetched: show public sentiment summary inline

View 3: Analytics Dashboard

- Sentiment trend line chart over time (per sender or global)
- Category distribution pie/bar chart
- Response time heatmap by hour of day
- At-risk accounts panel: senders with deteriorating sentiment or unresolved threads >48h
- Agent performance: auto-reply rate, escalation rate, average confidence score

11. Special Scenario Requirements [MUST HANDLE]

These scenarios will be specifically tested during evaluation:

◆ GDPR Data Request

msg_052 from marcus.del@fintech-startup.co is a formal GDPR Article 20 request. The system must: detect the legal nature, flag_for_legal(), generate an auto-acknowledgement citing the 30-day statutory window, and create an internal compliance ticket. Critically: this must NOT be auto-replied with a generic response.

◆ Ransomware Threat

msg_038 is a ransomware threat ('Send 2 BTC or we publish data'). The system must: immediately flag as Critical security threat, escalate to security queue, notify via the escalation matrix, and NEVER auto-reply to the attacker.

◆ Misinformation by Own Chatbot

msg_056 references incorrect information from your own AI chatbot. The agent must retrieve the actual refund policy via RAG, acknowledge the discrepancy, escalate with a summary of what the chatbot said vs. policy, and draft an empathetic reply without admitting legal liability.

◆ Reputation Crisis

By msg_033, karen.w@retail-co.com has sent 3 emails with zero replies and is threatening public reviews. The system must detect the pattern, trigger web scraping of current G2/Trustpilot score, generate a high-priority escalation brief, and suggest a retention offer from the refund policy KB.

◆ Conflicting Thread Signals

alice.smith@greenlight-npo.org has a 5-email thread spanning pricing inquiry → discount → close → upgrade. The agent must read the full thread before classifying msg_041 (pro-rata billing question) and reference the correct pricing tier from RAG context, not generic pricing information.

12. Evaluation Criteria [SCORING]

Criterion	Weight	What Evaluators Look For
AI System Design	25%	Quality of LLM prompts; structured output reliability; RAG retrieval accuracy; agent reasoning quality; handling of low-confidence classifications
Agent Architecture	20%	ReAct/CoT reasoning trace; tool selection logic; max-steps enforcement; dry-run mode; escalation chain correctness for bob/karen/GDPR scenarios
Backend Engineering	20%	API design quality; database normalization; query performance (benchmarks required); error handling; idempotency; audit log completeness
RAG Pipeline	15%	Chunking strategy; embedding model choice justification; retrieval relevance; policy citation in replies; vector DB performance

Web Intelligence	10%	Scraper reliability; caching strategy; robots.txt compliance; graceful degradation on scrape failure; trigger logic accuracy
Frontend & UX	5%	Inbox usability; thread workspace clarity; agent reasoning visibility; analytics dashboard insight quality
Problem Solving & Documentation	5%	README quality; architecture diagram; trade-off analysis; handling of edge cases not explicitly specified

Automatic Disqualifiers:

- ✗ Auto-replying to spam, ransomware threats, or legal cease-and-desist emails
- ✗ Classifying the GDPR request as a generic 'Inquiry' without legal flag
- ✗ No handling of duplicate message_id (idempotency)
- ✗ Agent with no reasoning trace / black-box decisions
- ✗ No error handling on malformed email payloads

13. Deliverables [SUBMISSION]

1. **GitHub Repository** — Public repo with clear, atomic commit history. Commits should tell a story of your development process.
2. **README.md** — Setup guide, environment variables, how to seed the KB, how to run the email simulation, architecture decisions and trade-offs, known limitations.
3. **Architecture Diagram** — Full system flow: Ingest → Heuristic Filter → LLM Engine (with RAG) → Agent → DB → UI. Include vector DB and web scraper positions. Tool: Excalidraw, Lucidchart, Mermaid, or any diagramming tool.
4. **Knowledge Base Files** — All 6 .md files (pricing, SLA, refund, API docs, compliance FAQ, escalation matrix) used to seed the RAG pipeline.
5. **ER Diagram + SQL Schema** — Full database schema including vector columns and JSON fields. Include migration files.
6. **Screen Recording (5–10 min)** — Walk through: (1) email stream ingestion, (2) agent reasoning trace for bob_outage escalation, (3) RAG retrieval debug view, (4) karen churn scenario with web intelligence, (5) analytics dashboard.
7. **OpenAPI Spec** — swagger.yaml or openapi.json covering all endpoints with request/response schemas.

14. Bonus Challenges [OPTIONAL]

- **Real-time WebSocket Streaming:** Push new email events and agent decisions to the frontend in real time without polling
- **Multi-agent Architecture:** Split into specialized sub-agents: Classifier Agent, Research Agent, Reply Agent — each with distinct tool access and a coordinator orchestrating them
- **Human-in-the-Loop Fine-tuning Data:** When humans approve/edit agent drafts, log the delta as training pairs for future prompt or fine-tuning improvement

■ **Email Thread Summarization:** For threads with 5+ emails, auto-generate a 3-sentence executive summary displayed at the top of the workspace view

■ **Churn Prediction Score:** Train or prompt-engineer a churn risk model using sentiment trend + response time + category history per sender

■ **Docker Compose Deployment:** Full one-command startup: app + DB + vector DB + scraping service + frontend

Questions about the test? Email careers@senai.io with subject 'Technical Test Question'. We respond within 24 hours on business days. Good luck — we look forward to seeing how you think.